

Portable Agent Harness Architecture

A Capability-Centric Framework for Governed AI Ecosystems in Sovereignty-Bounded Enterprises

Revision 2.2

James (JD) Longmire

Northrop Grumman Fellow (unaffiliated research)

Chief Architect, Digital Ecosystems

ORCID: 0009-0009-1383-7698

Correspondence: jdlongmire@outlook.com

February 2026

Abstract

The current generation of enterprise AI deployments is organized around assistants. A chatbot, a copilot, a vendor-specific orchestration runtime, or a single foundation-model interaction surface becomes the architectural anchor, and governance, identity, and execution control are bolted on around it. This paper argues that this assistant-centric pattern is structurally inadequate for enterprises operating under sovereignty constraints, multi-vendor obligations, or cross-enclave security boundaries. It proposes an alternative pattern, Portable Agent Harness Architecture (PAHA), in which the durable architectural layer is a meta-harness providing centralized governance, bounded execution, and substrate arbitration over which fit-for-purpose operational consoles and composable agents are instantiated. The paper develops the architectural primitives, specifies the minimum viable harness implementation, decomposes the harness into three interacting planes (governance, cognitive, operational), works through a cross-domain scenario, maps the framework to existing enterprise architecture standards, and proposes a phased adoption path for organizations with vendor-native AI deployments already in flight. It concludes with the qualified market thesis that orchestration supremacy applies most strongly to organizations whose security posture or regulatory regime prevents acceptance of vendor-integrated orchestration, and identifies the practitioner experience problem as a precondition for governance to function rather than an output of it. Appendix A presents a deployment view reference architecture for organizations operating across multiple security enclaves. An implementation specification subordinate to a parent enterprise systems engineering management plan is in development and will cite this paper as its architectural framework.

1. Introduction

Enterprise AI adoption in 2026 is characterized by parallel deployments of vendor-specific copilots and orchestration runtimes. A typical large enterprise may simultaneously operate Microsoft 365 Copilot, AWS Bedrock AgentCore, Cisco AI Canvas, ServiceNow Now Assist, and several program-specific assistants. Each carries its own identity model, audit format, policy framework, execution boundary semantics, and governance artifacts. Each vendor advances its orchestration layer along an independent roadmap. The aggregate result is fragmentation rather than capability.

This paper develops an alternative architectural pattern. The argument proceeds in three movements. The first establishes the structural inadequacy of assistant-centric architecture for enterprises with sovereignty, multi-vendor, or cross-enclave constraints. The second develops the meta-harness as a durable architectural layer with three interacting planes, specifies what a minimum viable harness implementation comprises, and works through a concrete cross-domain scenario. The third applies the framework to fit-for-purpose operational consoles, maps it to existing enterprise architecture standards, proposes a phased adoption path, and concludes with a qualified market thesis about the long-term durability of orchestration as a value layer.

The argument is grounded in defense IT operational realities. Specific examples draw from aerospace and defense environments because that domain forces explicit treatment of constraints that commercial deployments often elide: classified network segmentation, ITAR and EAR boundaries, FedRAMP and CMMC compliance, ATO cycles, and air-gapped deployment requirements. The architectural pattern generalizes beyond defense, but the constraints make the pattern's value visible.

This paper is written for enterprise architects, governance and compliance leaders, and senior IT decision-makers in organizations operating under sovereignty constraints, multi-vendor obligations, or cross-enclave security boundaries. It is not written for organizations with single-vendor consolidation strategies in commercial environments; for those organizations, vendor-integrated orchestration may be the more economically rational choice, and the architectural pattern proposed here represents unnecessary overhead. Section 14 develops this scope distinction explicitly. Readers should approach the paper with their own organizational context in mind: the framework's value is conditional on the constraints it addresses.

2. The Limits of Assistant-Centric Architecture

The assistant-centric pattern conflates three concerns that should be architecturally separable. The interaction surface (how a user engages the system), the orchestration runtime (how reasoning, tool invocation, and memory are coordinated), and the governance framework (how identity, policy, audit, and provenance are enforced) are bound together in vendor-native implementations. A user of a vendor copilot cannot select a different governance model without abandoning the assistant. An enterprise that adopts multiple copilots inherits multiple governance models that must be reconciled at the integration layer rather than the architectural layer.

This conflation produces specific failure modes. Identity becomes vendor-scoped rather than enterprise-scoped. Audit trails fragment across orchestration runtimes with incompatible schemas. Policy enforcement becomes a per-vendor configuration exercise rather than a centralized governance function. Memory and context governance bind to the vendor's storage substrate. Tool invocation patterns are constrained by the vendor's plugin or function-calling model. Each of these failures is recoverable in isolation. The aggregate is not.

For enterprises operating in commercial environments with single-vendor consolidation strategies, the assistant-centric pattern may be acceptable. The governance debt is contained, the integration burden is manageable, and the vendor's roadmap aligns with the enterprise's needs by definition because the enterprise has accepted that alignment as a strategic decision. For enterprises operating under sovereignty constraints, multi-vendor obligations, or cross-enclave security boundaries, the assistant-centric pattern is structurally inadequate.

The structural shift from assistant-centric to capability-centric reframes five concerns that the assistant-centric pattern bundles. Table 1 summarizes the contrast.

Concern	Assistant-centric pattern	Capability-centric pattern
Identity scope	Vendor-scoped; each copilot maintains its own identity context, with enterprise IAM mapped at integration time.	Enterprise-scoped; the harness enforces a single identity model across all consoles and agents within an enclave.
Audit format	Fragmented; each orchestration runtime emits its own schema, requiring reconciliation for cross-tool audit.	Unified; a shared audit schema flows through the harness audit bus regardless of which substrate handled the work.
Policy placement	Per-vendor configuration; each copilot's policy framework must be configured separately and kept in sync.	Centralized at the governance plane; one policy framework applies across all consoles, agents, and substrates.
Substrate flexibility	Bound to vendor's substrate; substrate change requires assistant migration or vendor switch.	Substrate-arbitrated; per-agent substrate selection through the capability registry. Substrate changes are adapter operations.
Cross-enclave behavior	Per-vendor cross-enclave story varies; classified deployment often requires a different vendor than unclassified.	Pattern-portable; the harness pattern instantiates per enclave with shared governance artifacts. The instance varies; the pattern does not.

Table 1. Five concerns that the assistant-centric pattern bundles and the capability-centric pattern separates.

3. Architectural Primitives of the Meta-Harness

Sections 3 through 5 develop the architecture in three steps. Section 3 specifies the architectural primitives that the harness pattern depends on. Section 4 specifies the seven minimum viable harness services that realize those primitives operationally. Section 5 organizes the services into three architectural planes that distinguish governance from cognition from operation. The progression is from abstract primitive to concrete service to organizational structure. A reader who follows the progression should understand the framework's structural shape by the end of Section 5.

The PAHA pattern proposes a meta-harness as the durable architectural layer. The harness is not an assistant. It is the substrate over which assistants, agents, and consoles are instantiated. Its primitives are governance contracts, execution boundaries, substrate adapters, and capability registries.

A governance contract — *the declarative specification of identity, policy, audit, and provenance obligations that any agent or console operating within the harness must satisfy.* The contract is substrate-agnostic at the policy layer, with substrate-specific enforcement delegated to the cognitive plane through adapters.

An execution boundary — *a structural constraint on agent action enforced at the harness layer regardless of the agent's reasoning.* Boundaries are not trust levels and they are not advisory. They are enforced at the plane where the boundary is physically meaningful.

Execution boundaries fall into three classes, each enforced at a different plane. This taxonomy matters because PAHA's safety properties depend on enforcing the right boundary at the right plane, not on enforcing all boundaries everywhere.

Data-scope boundaries constrain what an agent may read. They are enforced primarily at the operational plane (through adapters to data stores, indexes, and enterprise systems) with policy and identity constraints flowing down from the governance plane. A data-scope boundary that says “this agent may not read CUI marked content” is enforced by the sensitivity-label-aware adapter to the document store, not by the agent's reasoning about what it should look at.

Action-scope boundaries constrain what an agent may write or invoke. They are enforced at the operational plane through adapters to systems of record, ticketing platforms, and infrastructure control planes. An action-scope boundary that says “this agent may not modify production firewall rules” is enforced at the network-control adapter, not at the cognitive plane through prompt engineering.

Trust and autonomy boundaries constrain the level of human approval required before an agent's recommendations become actions. They are enforced at the governance plane through the trust escalation model. A trust boundary that says “this agent operates in advisory mode only” is enforced by the governance plane refusing to forward recommendations as actions, regardless of how confident the cognitive plane is in its output.

A trust escalation model — *the governance-plane primitive that defines the levels of agent autonomy (typically advisory, recommend-and-approve, automated-with-audit, fully autonomous) and the criteria required to transition between them.* Under PAHA, this model is enforced uniformly across all consoles and substrates within an enclave.

A substrate adapter — *the layer that mediates between the harness and a specific cognitive substrate (a foundation model, a symbolic engine, or a hybrid system).* The adapter translates harness primitives into substrate-specific invocations and translates substrate outputs into harness-governed signals. Adapters make substrates interchangeable at the harness layer.

A capability registry — *the catalog of agents, tools, and consoles available within the harness, along with their governance contracts, execution boundaries, and substrate requirements.* The registry is the mechanism through which composability is realized and through which substrate bindings are made replaceable.

4. Minimum Viable Harness Implementation

The architectural primitives of Section 3 are necessary but not sufficient. An organization adopting PAHA must build, acquire, or compose a real implementation that exposes those primitives as operational

services. This section specifies the seven services that constitute a minimum viable harness. An implementation that omits any of these services is not a PAHA implementation; it is a partial pattern that will leak governance debt into the consoles built above it.

The seven services described below correspond to the architectural planes introduced in Section 5. Three services live primarily in the governance plane, two in the cognitive plane, and two in the operational plane. The console integration façade spans all three.

4.1 Identity and Policy Enforcement Gateway

The gateway is the entry point through which all agent invocations pass. It authenticates the calling identity (user, service, or upstream agent), evaluates the governance contract associated with the requested capability, and either authorizes or rejects the invocation. The gateway is not optional and it is not a wrapper around vendor authentication. It is the canonical authority for identity and policy decisions across all consoles within the harness instance. In defense IT contexts, the gateway integrates with existing IAM infrastructure (PIV/CAC, Entra ID Government, Active Directory Federation Services) and applies the enterprise's existing policy framework to harness operations rather than substituting its own.

4.2 Capability Registry Service

The registry catalogs every agent, tool, and console available within the harness. Each entry specifies the governance contract under which the capability operates, the execution boundaries that constrain its action, the substrate it currently binds to, and the identities authorized to invoke it. The registry is the system of record for what the harness can do. New capabilities are added by registry registration, not by code deployment. Substrate changes are accomplished by updating the registry binding, not by rewriting the agent. Capability retirement is a registry operation. Without a registry, agents become brittle deployments rather than composable artifacts.

4.3 Audit Bus

The audit bus is the canonical pipeline through which all harness activity is recorded. Every gateway authorization decision, every capability invocation, every substrate call, every operational action, and every governance signal flows through the bus and is persisted in a tamper-evident log conforming to a shared schema across all consoles within the harness instance. The bus is what makes cross-console audit possible. It is also what enables enclave-level federation: harness instances in different enclaves can share aggregate audit signals without exchanging operational data, because the schema is common while the instance data is separated.

4.4 Substrate Adapter Layer

The adapter layer is the boundary between the cognitive plane and specific cognitive substrates. Each adapter translates harness primitives into substrate-specific invocations (prompts, function calls, retrieval requests) and translates substrate outputs into harness-governed signals (responses, confidence scores,

refusal indicators, citations). The adapter layer is what makes substrate arbitration possible. An organization that wants to swap Claude for a local OSS model on a specific agent does so by changing the adapter binding in the registry, not by rewriting the agent. Adapters also enrich substrate-specific safety signals into governance-plane primitives where the organization has chosen to do so (see Section 11).

4.5 Execution Boundary Engine

The boundary engine is the runtime component that enforces data-scope, action-scope, and trust boundaries during agent execution. When an agent attempts to read a document, the boundary engine validates the read against the data-scope boundary in the agent's governance contract before the operational adapter is invoked. When an agent attempts to write to a system of record, the boundary engine validates the write against the action-scope boundary. When an agent's output transitions from recommendation to action, the boundary engine consults the trust escalation model and either forwards the action, queues it for approval, or rejects it. The boundary engine is the safety property of the harness. Its correctness is what makes the harness trustworthy in defense IT contexts.

4.6 Memory Governance Service

The memory service governs how agents store, retrieve, and forget context across sessions and across consoles. It enforces retention policies, classification handling for stored context, and isolation between agents that should not share memory. In defense IT contexts, the memory service is also where the cross-enclave boundary is most carefully managed: an agent operating in an unclassified console must not retain memory that could leak into a classified context, and the memory service is what enforces that constraint. Without a memory governance service, agents become accidental aggregators of sensitive context, and the harness's classification handling fails silently.

4.7 Console Integration Façade

The façade is the surface through which consoles invoke harness capabilities. It exposes a uniform invocation contract to console developers regardless of which substrate handles a particular capability, which adapter mediates a particular operational call, or which agent is being orchestrated. The façade is what makes consoles substrate-agnostic by construction. A console developer building the Enterprise Operations console does not need to know whether the Enterprise Architect Agent runs on Claude or on a local model; the façade abstracts that decision. This is the service that most directly enables fit-for-purpose console specialization without requiring console-specific governance.

5. The Three Architectural Planes

The seven harness services described in Section 4 are organized into three interacting planes. Each plane has distinct responsibilities and independent evolution paths.

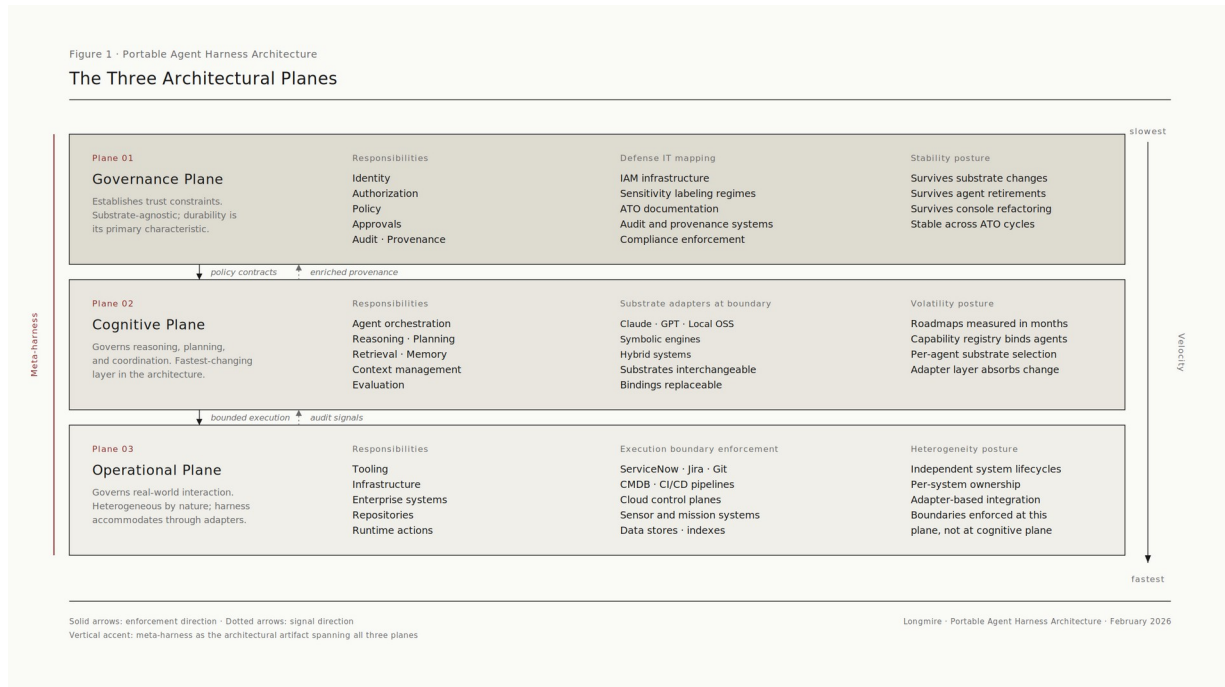


Figure 1. The three architectural planes of Portable Agent Harness Architecture. The governance plane establishes substrate-agnostic trust constraints and is the slowest-changing layer. The cognitive plane governs orchestration and reasoning and is the fastest-changing layer; substrate adapters live at its boundary. The operational plane governs heterogeneous real-world interaction. Solid arrows indicate enforcement direction; dotted arrows indicate signal direction. The vertical accent on the left indicates that the meta-harness spans all three planes as the durable architectural artifact.

Figure 1 organizes the architecture vertically by rate of change. The governance plane sits at the top because it must be the most stable. Substrate-specific changes in the cognitive plane below cannot be permitted to invalidate governance contracts. The operational plane sits at the bottom because that is where boundaries are physically enforced through adapters to enterprise systems. The harness pattern's value proposition rests on this rate-of-change separation: enterprise governance evolves on cycles measured in years, while cognitive substrates evolve on cycles measured in months.

5.1 Governance Plane

The governance plane is responsible for identity, authorization, policy, approvals, audit, provenance, and compliance enforcement. The identity and policy enforcement gateway, the audit bus, and the trust-escalation portion of the execution boundary engine all live here. The plane establishes the trust constraints under which all cognitive and operational activity occurs. Its primary characteristic is durability. Governance artifacts must survive substrate changes, agent retirements, and console refactoring. The governance plane is the slowest-changing layer in the architecture, and its stability is what makes the rest of the architecture portable.

In defense IT contexts, the governance plane corresponds to existing IAM infrastructure, sensitivity labeling regimes, audit and provenance systems, and ATO documentation. PAHA does not replace these

systems. It exposes them as the canonical governance substrate over which AI capabilities are instantiated.

5.2 Cognitive Plane

The cognitive plane is responsible for agent orchestration, reasoning, planning, retrieval, memory, context management, and evaluation. The capability registry service and the substrate adapter layer live here. The plane governs how cognitive work is decomposed, sequenced, and validated. Its primary characteristic is volatility. Foundation models, prompting techniques, retrieval methods, and evaluation frameworks evolve on roadmaps measured in months. The cognitive plane is the fastest-changing layer in the architecture, and the harness pattern's value is largely in protecting the rest of the enterprise from that velocity.

The substrate adapters live at the boundary between the cognitive plane and the harness. Different agents may run on different substrates. A capability registry binding declares which substrate an agent uses, but the binding is replaceable.

5.3 Operational Plane

The operational plane is responsible for tooling, infrastructure, enterprise systems, repositories, execution environments, and runtime actions. The data-scope and action-scope portions of the execution boundary engine, along with the memory governance service, live here. The plane governs real-world interaction. Its primary characteristic is heterogeneity. Operational systems have their own lifecycles, ownership, and integration patterns, and the harness must accommodate that diversity through adapters rather than impose a uniform model.

The operational plane is where execution boundaries are physically enforced. A boundary that says “this agent may not write to the production change management system” is enforced at the operational adapter to ServiceNow, not at the cognitive plane.

6. Capability-Centric Architecture

The shift from assistant-centric to capability-centric architecture is the framework's central reorientation. In the capability-centric view, the agent is not the system. The governed ecosystem is the system. Agents are ephemeral operational personas instantiated against role definitions, capability contracts, context scopes, and execution permissions.

This reorientation has practical consequences. Agent retirement becomes a registry operation rather than a system migration. Substrate changes become adapter operations rather than agent rewrites. New consoles become entries in the capability registry rather than parallel deployments with their own governance frameworks. The harness absorbs the variability that vendor-native architectures expose to the enterprise.

The reorientation also reframes the question of what to build. Under assistant-centric thinking, the question is “which copilot should we adopt?” Under capability-centric thinking, the question is “what governance contracts, execution boundaries, and capability registry must we maintain, and which agents and substrates should we instantiate against them?” The first question has a vendor-determined answer. The second has an enterprise-determined answer.

6.1 The Means and Mechanisms Distinction

Underlying the capability-centric reorientation is a deeper distinction that the assistant-centric pattern conflates. A *means* is purposive: it is what the system uses *toward* its mission within its constraints. Means carry teleological weight; their existence is justified by what they advance, not by what they can do in isolation. A *mechanism* is operationally agnostic: it is the machinery by which something works, independent of why. Mechanisms exist as available capacity. They become meaningful only when elected by some purposive layer; until then they are inventory.

Means and mechanisms describe the same underlying reality from two different stances. The same code execution environment may be inventoried as a mechanism (available infrastructure) and elected as a means (purposively in use as part of the system's repertoire toward its mission). The first description tells what is available; the second tells what is purposively in use. The assistant-centric pattern conflates the two: a vendor copilot ships a bundled set of mechanisms (its tools, its memory, its retrieval) and presents them as the system's means by default, without requiring purposive election. The capability-centric pattern separates them: mechanisms are inventoried in the operational plane and elected as means through purposive declaration in the governance plane. This separation is what makes substrate arbitration, capability composition, and cross-enclave portability possible.

The relationship between purposive layers and mechanism layers is *selection*, not abstraction. The harness pattern realizes this selection relation: governance contracts and capability registries declare which mechanisms are purposively elected as means; substrate adapters mediate between purposive declarations and operational mechanisms; the execution boundary engine enforces the selection at runtime. Mechanisms not elected by any purposive layer are not part of the system's disposition even if they remain operationally available. This is the architectural commitment that makes the framework governable: the question of what the system uses (mechanism inventory) is separable from the question of what the system is for (purposive disposition).

7. Fit-for-Purpose Operational Consoles

Once a meta-harness exists, specialized consoles can be instantiated as domain-specific operational surfaces over the same governed substrate. The harness remains stable. What changes between consoles is context scope, user interaction patterns, operational permissions, domain tooling, trust thresholds, and evaluation criteria.

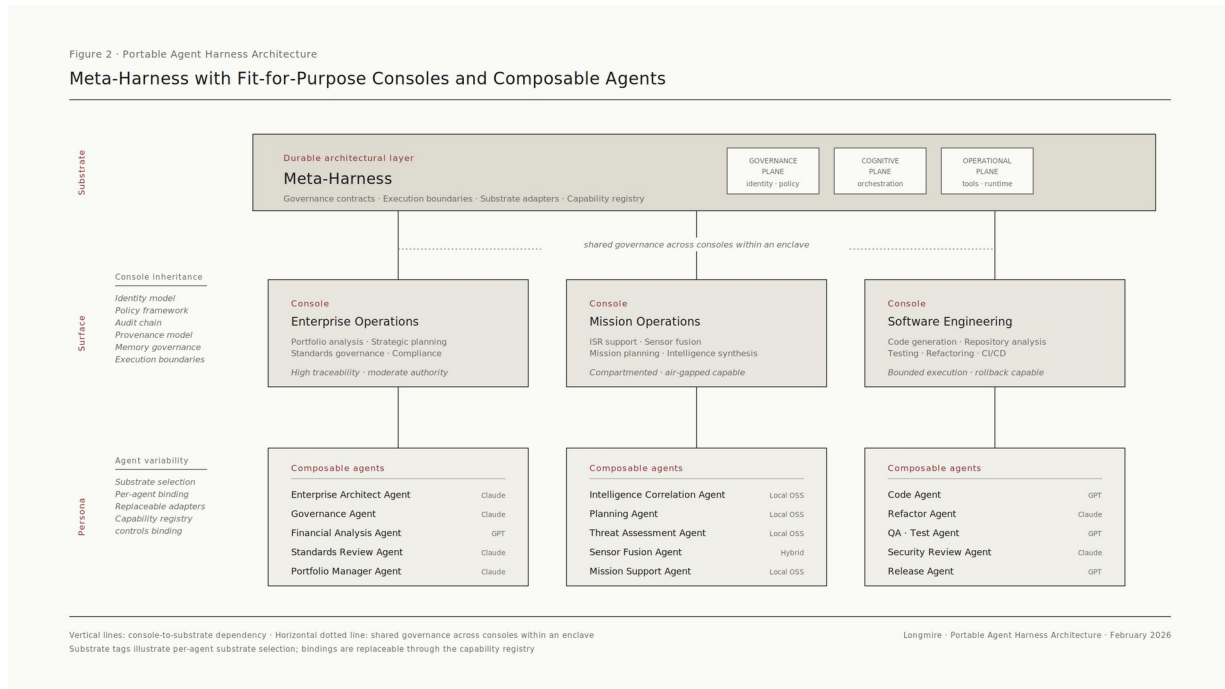


Figure 2. Meta-harness with fit-for-purpose consoles and composable agents. The meta-harness spans all three architectural planes (governance, cognitive, operational) and provides the durable substrate over which specialized consoles operate. Each console exposes a different operational surface optimized for its mission and user class while inheriting the same governance, audit, and execution boundary infrastructure. The horizontal dotted line denotes shared governance across consoles within an enclave. Substrate tags on each agent illustrate per-agent substrate selection through the capability registry; bindings are replaceable.

Figure 2 illustrates the structural relationship among the harness, the consoles, and the agents. The result is horizontal governance consistency combined with vertical operational specialization. The Enterprise Console and the Software Engineering Console expose entirely different operational surfaces, but they share the substrate that makes both governable. An Enterprise Architect Agent running on Claude and a Code Agent running on GPT operate under the same governance contract, the same audit chain, and the same execution boundary framework, even though their substrates and operational surfaces differ.

One clarification before this argument develops further. The phrase “meta-harness” in Figure 2 refers to a harness instance within a single enclave. In environments with hard security boundaries, harness instances do not span enclaves; the architectural pattern instantiates per enclave with shared governance artifacts at the schema layer. Section 9 develops this constraint in full. Readers should hold the phrase “one harness, many surfaces” as architectural shorthand rather than as a literal claim about a single instance bridging classification boundaries.

7.1 Enterprise Operations Console

Typical functions include portfolio analysis, strategic planning, standards governance, document synthesis, business capability mapping, and compliance analysis. Representative agents include an Enterprise Architect Agent, a Governance Agent, a Financial Analysis Agent, and a Standards Review

Agent. The primary characteristics are high traceability, moderate execution authority, and strong audit requirements. Outputs are typically advisory artifacts that feed leadership decision-making.

7.2 Mission Operations Console

Typical functions include ISR support, sensor fusion, mission planning, operational analysis, and intelligence synthesis. Representative agents include an Intelligence Correlation Agent, a Planning Agent, a Threat Assessment Agent, and a Sensor Fusion Agent. The primary characteristics are compartmented context, strict authorization controls, deterministic escalation, and air-gapped deployment support. Outputs feed mission decisions and require provenance sufficient to support after-action review.

7.3 Software Engineering Console

Typical functions include code generation, repository analysis, testing, refactoring, and CI/CD orchestration. Representative agents include a Code Agent, a QA Agent, a Security Review Agent, and a Release Agent. The primary characteristics are bounded execution, rollback capability, ephemeral runtime isolation, and strong policy enforcement. Outputs are typically code artifacts and pipeline operations that traverse standard development governance gates.

The point of cataloging these consoles is not exhaustiveness. The point is that the harness pattern accommodates domain-specific operational characteristics without requiring domain-specific governance frameworks. The same identity model, the same audit chain, the same policy framework, and the same provenance model apply across all three.

8. A Worked Scenario: Cross-Domain Compliance Query

To make the harness's distinctive value concrete, consider a scenario that vendor-native architectures handle poorly: a compliance officer working in the Enterprise Operations console needs to verify that a specific Mission Operations data source meets retention requirements specified in an unclassified compliance framework. The data source itself contains compartmented information that the compliance officer is not cleared to view in raw form, but the metadata required to verify retention compliance (creation timestamps, classification levels, retention markers) can be released under controlled redaction.

This scenario fails under assistant-centric architecture. A vendor copilot in the unclassified Enterprise Operations console cannot read the Mission data source. A vendor copilot in the Mission console cannot return data to the unclassified context. The compliance officer ends up doing manual coordination across two human teams operating in two enclaves, which is the workflow PAHA is supposed to obsolete.

Under PAHA, the scenario unfolds as follows. At a glance, the sequence is:

1. Compliance officer invokes a Compliance Verification Agent in the Enterprise Operations console; the gateway authenticates and authorizes.

2. Capability registry binds the agent to its substrate; substrate adapter decomposes the query into retrieval and evaluation subtasks.
3. Execution boundary engine enforces data-scope boundary; classification-aware metadata is releasable, content is not.
4. Mission-side adapter returns metadata only; the cross-enclave metadata flow is governed by the agent's contract, not by ad-hoc bridge logic.
5. Cognitive plane evaluates metadata against compliance framework and produces a finding under advisory trust posture.
6. Audit bus records the full chain across both enclaves' audit aggregators under the shared schema; the compliance officer receives the finding with full provenance.

The detailed unfolding follows. Each step below corresponds to one entry in the sequence above.

The compliance officer invokes a Compliance Verification Agent in the Enterprise Operations console. The console integration façade routes the invocation through the identity and policy enforcement gateway. The gateway authenticates the officer's identity (PIV credential, role: Compliance Officer, clearance level: Secret, console: Enterprise Operations Unclassified) and evaluates the agent's governance contract. The contract authorizes the invocation.

The capability registry binds the Compliance Verification Agent to its current substrate (Claude, in this example). The substrate adapter translates the officer's natural-language query (“verify retention compliance for data source MDS-4471 against framework FR-2024-08”) into a structured invocation. The cognitive plane decomposes the query into two subtasks: retrieve metadata from MDS-4471, and evaluate that metadata against FR-2024-08.

The execution boundary engine evaluates the data-scope boundary for the metadata retrieval. The boundary specifies that the Compliance Verification Agent may read classification-aware metadata fields from Mission data sources but may not read content fields. The operational plane invokes the Mission-side adapter, which is itself a federated boundary: the adapter operates within the Mission enclave's harness instance and returns only metadata fields, with classification markers preserved. The metadata crosses the enclave boundary because it is releasable under the agent's data-scope contract; the content does not because it is not.

The cognitive plane evaluates the metadata against the compliance framework. The substrate produces a finding: “MDS-4471 satisfies retention requirements under FR-2024-08; creation timestamp 2023-04-12, last review 2025-11-03, retention marker R-7Y, classification S//NF.” The execution boundary engine evaluates the action-scope boundary for the response: the agent may return findings to the requesting console under appropriate redaction. The trust escalation model applies advisory mode (the agent's compliance findings are recommendations, not adjudications).

The audit bus records the full chain: identity, invocation, governance contract evaluation, substrate selection, cross-enclave metadata retrieval with classification handling, finding generation, and response

routing. The audit record is queryable from the Enterprise Operations enclave's audit aggregator and federated to the Mission enclave's audit aggregator under the shared schema. The compliance officer receives the finding. The metadata never leaves the harness's controlled redaction. The Mission content never crosses the enclave boundary. The compliance verification, which previously required two-team coordination across two enclaves, completes in a single agent invocation with full provenance.

This scenario is what PAHA exists to enable. It requires every architectural primitive: governance contracts (the Compliance Verification Agent's contract specifying what it may do), execution boundaries (data-scope, action-scope, and trust boundaries enforced at the appropriate planes), substrate adapters (the cognitive substrate selection and the cross-enclave operational adapter), the capability registry (binding the agent to its substrate and contract), and the audit bus (federating provenance across enclaves). It also requires every harness service: gateway, registry, audit bus, adapter layer, boundary engine, memory governance (the agent must not retain Mission metadata in unclassified-context memory), and console integration façade. A scenario this dense in cross-cutting requirements is exactly the case where assistant-centric architectures fail and PAHA's investment is justified.

9. The Enclave Constraint

The architecture as described above must be qualified for environments with hard security boundaries. The claim that “all consoles inherit the same identity model” is true at the level of pattern and schema. It is not true at the level of instance.

An Enterprise Console operating on unclassified GovCloud cannot share an identity store with a Mission Console operating on a classified network. The identity store cannot cross the boundary. What can cross the boundary is the identity schema, the identity governance pattern, and the audit format. The harness, in this environment, is not a single instance spanning enclaves. It is a pattern instantiated separately per enclave, with shared governance artifacts at the schema and policy layer.

This distinction matters because the framework's value proposition depends on it. Horizontal governance consistency does not require that a single harness instance span all enclaves. It requires that the governance pattern be portable across enclaves and that the artifacts produced in each enclave conform to a shared schema. The harness pattern, as an architectural concept, transits security boundaries. The harness instance does not.

In practice, this means an enterprise operating in defense IT will have at minimum a harness instance for unclassified GovCloud workloads, a harness instance for classified workloads, and potentially additional instances for partner-specific or program-specific enclaves. Each instance enforces the same governance contracts. Each is independently authorized. Federation occurs at the schema and audit-aggregation layer, with no operational data crossing the boundary.

The language of “one harness, many surfaces” is therefore a simplification that should be tightened to “harness-as-pattern, instantiated per enclave with shared governance artifacts.” The architectural value is preserved. The operational claim is more defensible.

10. Shared Governance Across Specialized Ecosystems

Within an enclave, the harness pattern enables governance reuse across specialized consoles. This is the framework's strongest operational claim, and it warrants explicit articulation.

All consoles within a harness instance inherit the same identity model, the same policy framework, the same audit architecture, the same provenance chain, the same memory governance model, and the same execution boundary enforcement. This produces horizontal governance consistency combined with vertical operational specialization. The Enterprise Console and the Software Engineering Console expose entirely different operational surfaces, but they share the substrate that makes both governable.

The avoided failure mode is fragmentation into isolated AI silos, each with its own governance approach, each requiring its own ATO, each carrying its own audit format, and each subject to its own vendor roadmap volatility. Fragmentation is the default outcome of assistant-centric architecture. Shared governance is the default outcome of harness architecture, conditional on the harness being implemented as a real architectural artifact rather than a paper abstraction.

The implementation cost of this shared governance is non-trivial. The harness must be built, maintained, and evolved by the enterprise (or by a harness vendor distinct from the foundation model and console vendors). The cost is justified by the avoided fragmentation, but the trade-off must be made explicit. Organizations without the architectural maturity to maintain a harness should not adopt PAHA. They should instead consolidate on a single vendor's orchestration runtime and accept the resulting governance constraints.

Three illustrative indicators help distinguish organizations with sufficient architectural maturity from those without. First, an existing enterprise architecture governance function with real authority over technology decisions; if the EA function is advisory rather than authoritative, the harness's centralized governance commitments will not be enforceable. Second, an operating CMDB or equivalent configuration management discipline that produces accurate, current records of what is deployed; without it, the harness cannot reliably know what mechanisms are available for election as means. Third, a central IAM that is actually authoritative across the enterprise rather than one identity provider among several; the harness's identity model depends on enterprise-scoped identity rather than vendor-scoped identity, and vendor-scoped fragmentation in IAM will propagate into the harness if not resolved first. These indicators are illustrative rather than strict criteria; an organization missing one but strong on the other two may still be viable. An organization missing all three should consolidate on vendor-integrated orchestration and revisit the harness pattern when the indicators mature.

10.1 Runtime Assurance as an Architectural Commitment

One class of governance properties deserves explicit treatment because it distinguishes harness architectures from vendor-native architectures most sharply. Runtime assurance — the continuous monitoring of agent behavior for drift from declared mission, coherence between action sequences and stated objectives, deviation from execution policy, and explainability of decisions — is an architectural

commitment the harness pattern enables and that vendor-native architectures cannot enforce uniformly. The reason is structural rather than incidental. Vendor-native runtime monitoring is bounded by what the vendor instruments and what the vendor's telemetry surfaces; cross-vendor uniformity in runtime monitoring is achievable only through reconciliation at the integration layer, which reproduces the same fragmentation that the harness pattern exists to resolve.

Under the harness pattern, runtime assurance becomes a first-class governance property: drift detection, coherence monitoring, deviation detection, and explainability surfacing apply uniformly across all consoles, agents, and substrates within an enclave because they are enforced at the harness layer rather than the vendor layer. This is not free; the harness must implement or integrate the runtime assurance instruments, and the assurance signals must be calibrated against operational baselines that themselves require engineering investment. But the resulting governance property is not achievable in vendor-native architectures regardless of integration effort, because the underlying runtime visibility is not uniform across vendors. For sovereignty-bounded enterprises whose regulatory or mission posture requires continuous evidence of governance conformance, runtime assurance as architectural commitment is one of the harness pattern's strongest justifications.

11. Dynamic Cognitive Substrate Arbitration

The harness allows substrate selection to be made per agent, per task, or per context, rather than per platform. This capability is architecturally elegant and operationally significant in environments where different cognitive substrates are required for different mission contexts.

The user does not necessarily know which substrate handled which task. The harness arbitrates. The operational workflow remains stable as substrates evolve, are deprecated, or are replaced. An `ArchitectureAgent` may run on Claude, a `CodeRefactorAgent` on GPT, an `AirGappedIntelAgent` on a local OSS model, and a `FormalVerificationAgent` on a hybrid symbolic-LLM system. The bindings are declared in the capability registry and are replaceable.

The framework must, however, address a circularity concern in the governance plane. If the harness selects a substrate based on mission fit, and the substrate's outputs feed into the governance plane's audit and policy framework, then the governance plane must be substrate-agnostic at the policy layer. This is straightforward when policy is declarative: allow and deny rules, sensitivity label enforcement, identity-based access controls, and approval requirements can all be expressed without reference to substrate-specific behavior.

The challenge appears when policy involves substrate-specific behaviors such as hallucination detection, alignment evaluation, or output safety filtering. Different substrates have different failure modes. A governance plane that abstracts over them either degrades to lowest-common-denominator policy, or it carries substrate-specific policy logic that breaks the abstraction. PAHA must explicitly choose between two architectural options.

The first option accepts lowest-common-denominator policy at the governance plane and pushes substrate-specific safety into the cognitive plane. The governance plane enforces only what all substrates can support. Substrate-specific safety enrichment occurs at the adapter layer and is reported to the governance plane as enriched audit signals rather than as policy primitives.

The second option defines an explicit substrate adapter interface that translates substrate-specific signals (hallucination scores, alignment metrics, refusal patterns) into governance-plane primitives. The governance plane carries a vocabulary of safety signals, and adapters map their substrate's outputs into that vocabulary. This preserves substrate-specific safety as governance information rather than relegating it to the cognitive plane.

The first option is simpler and more durable. The second is more expressive but accumulates governance complexity as substrates evolve. The choice between them is an enterprise architectural decision that depends on the organization's risk tolerance and the maturity of its governance plane. PAHA does not prescribe one over the other. It requires that the choice be made explicitly and documented as a constraint on the framework's instantiation.

To make the abstraction concrete, consider two substrates with substantively different safety signal vocabularies. A general-purpose conversational substrate may surface refusal patterns, calibrated confidence scores, and content-policy classifications as native safety signals. A domain-specialized substrate operating with structured output schemas (a planning system, a formal verification engine, a retrieval-grounded synthesis service) may surface schema-validity flags, retrieval grounding scores, and constraint-satisfaction proofs instead. Both substrates can be governed, but the safety signals are not interchangeable. Under the first option, both substrates' signals are absorbed by the cognitive plane's adapter layer and reported to governance only as enriched audit metadata; the governance plane's policy primitives stay the same. Under the second option, the governance plane carries an extended vocabulary that accepts both refusal patterns and schema-validity flags as first-class safety primitives, and adapters map each substrate's native signals into the appropriate vocabulary entries. The first option is what most defense IT enterprises will adopt initially because it minimizes governance plane churn; the second option becomes attractive when the substrate population stabilizes enough that the extended vocabulary is itself maintainable.

12. The Practitioner Experience as a Governance Precondition

The framework's strongest claim is that governance becomes centralized while operational specialization is preserved. The framework's most underdeveloped claim is that this centralization is invisible to the practitioner using a console.

Operations engineers, security analysts, and service desk staff do not experience the ecosystem. They experience the agent or console they are using. Governance value is invisible to them when it works correctly, and they will route around it if it adds friction. The vendor copilot that requires no enterprise

governance integration will be used in preference to the harnessed agent that requires authentication, sensitivity-label compliance, and audit logging if the enterprise harness imposes friction the vendor does not.

This is not a marginal concern. It is the dominant failure mode of governance frameworks in enterprise IT. A governance framework that practitioners abandon is worse than no framework, because the abandonment is silent and the resulting shadow IT is harder to detect than green-field deployment.

PAHA must therefore treat practitioner experience as a precondition for governance to function rather than as an output of it. Three implications follow.

First, the harness must be at least as fast and at least as capable as the vendor alternatives at the user-facing layer. If a harnessed agent takes ten seconds to respond to a query that the vendor copilot answers in two, practitioners will use the copilot. The harness's governance value is not visible to them, and the friction is. This requires real architectural commitment to user-experience parity: investment in harness runtime infrastructure, in cognitive plane optimization, and in adapter-layer performance tuning sufficient to match the parity that vendors achieve through their own dedicated investment. Treating user-experience parity as an architectural goal rather than an emergent property is the discipline that separates harness implementations practitioners adopt from harness implementations practitioners route around.

Second, the harness must absorb the integration burden that the vendor would otherwise expose to the user. Authentication, classification handling, audit metadata, and policy compliance must be enforced at the harness layer in ways that the user does not experience as procedural friction. Single sign-on, automatic sensitivity labeling, and transparent audit are not optional features. They are preconditions for adoption.

Third, the workforce adoption plan associated with PAHA deployment must include explicit measurement of practitioner trust, friction, and bypass rates. If practitioners are bypassing the harness, the harness has failed regardless of how elegant its architecture is. The same workforce adoption discipline that applies to vendor copilot deployments applies to harness deployments, with the additional requirement that the harness must demonstrate user-experience parity or superiority before broad deployment.

13. Tool Sprawl Reduction

A practical motivation for harness architecture is the reduction of tool sprawl. Enterprises currently accumulate separate copilots, separate orchestration stacks, separate vector databases, separate policy frameworks, separate memory systems, and separate plugin ecosystems. Each accretion increases governance debt, integration cost, and workforce confusion.

The harness pattern collapses this fragmentation. One harness instance per enclave, supporting many operational surfaces, replaces N vendor stacks. Vector storage, memory governance, retrieval infrastructure, and tool registries become harness services rather than per-vendor implementations. The substrate becomes interchangeable behind the harness.

This collapse is not free. The harness must implement or integrate the services it consolidates. An enterprise that adopts PAHA without building or acquiring a harness implementation has not reduced sprawl. It has added another layer to the existing stack. The architectural value depends on the harness being a real artifact under enterprise control or under a harness vendor's control whose roadmap aligns with the enterprise's needs.

For enterprises with sufficient architectural maturity to maintain a harness, the sprawl reduction is substantial. For enterprises without that maturity, the harness pattern is aspirational rather than operational, and the practical advice is to consolidate on a single vendor's orchestration runtime rather than attempt harness construction.

14. The Orchestration Layer as Durable Value: A Qualified Thesis

The strong form of the orchestration thesis holds that the long-term enterprise AI market will converge on a separation between foundation model providers, orchestration providers, governance providers, execution boundary providers, and domain-specific agent ecosystems. In this view, the durable value layer is not the foundation model. The durable layer is orchestration, governance, portability, trust enforcement, execution control, and ecosystem composability.

The strong form is plausible but premature. The market evidence is mixed. Anthropic, OpenAI, and Google are each building orchestration directly into their model-serving platforms through Model Context Protocol, function calling APIs, and structured tool use. AWS Bedrock AgentCore, Microsoft Copilot Studio, and Cisco AI Canvas are each foundation-model-adjacent orchestration layers tied to their respective ecosystems. The bet that orchestration becomes the durable value layer assumes these vertical integrations do not succeed in capturing the orchestration market.

The historical analogy worth examining is virtualization. In the early 2000s, the same argument was made: hypervisors would be the durable layer, and operating system vendors would commoditize. That partially happened, with VMware extracting substantial value from the hypervisor layer for over a decade. It also partially did not happen. Cloud providers absorbed the hypervisor layer into their stack, and the durable value moved up to platform services. The orchestration thesis could resolve either way.

The qualified form of the thesis is more defensible. Portable orchestration becomes a durable value layer for organizations whose security posture or sovereignty requirements prevent them from accepting vendor-integrated orchestration. This applies to defense, intelligence, regulated finance, and some healthcare contexts. It does not apply universally. For commercial enterprises with single-vendor consolidation strategies, vendor-integrated orchestration may be the more economically rational choice, and the harness pattern may represent unnecessary architectural overhead.

The qualified thesis carries practical implications for vendor selection. Organizations in sovereignty-bounded contexts should treat foundation model providers, orchestration providers, and governance providers as separable procurement decisions, even when a single vendor offers all three. The bundling

discount may be illusory if it forecloses the substrate arbitration that future operational requirements demand. Organizations in commercial contexts should make the bundling decision deliberately, with explicit acknowledgment of the architectural trade.

It is worth stating explicitly what would falsify the qualified thesis. Three conditions, if met, would indicate that vendor-integrated orchestration is sufficient even for sovereignty-bounded enterprises and that the harness pattern's investment is unjustified. First, vendor-integrated orchestration platforms acquire credible cross-enclave governance primitives — uniform policy enforcement across classified and unclassified deployments, schema-portable audit aggregation, identity-portable trust escalation — without requiring enterprise-side harness construction. Second, the foundation model market consolidates to the point where substrate arbitration ceases to be operationally meaningful, because the substrate diversity that motivates arbitration no longer exists. Third, the regulatory environment for defense IT and other sovereignty-bounded contexts evolves to accept vendor-attested governance in lieu of enterprise-controlled governance, removing the regulatory motivation for centralized harness control. None of these conditions appears imminent at the time of this writing, but architects evaluating PAHA against alternatives should monitor each. The qualified thesis is conditional on the persistence of the conditions that make it sound, and architectural commitments should be revisited when those conditions change.

15. Mapping to Existing Architecture Frameworks

PAHA is not a replacement for existing enterprise architecture frameworks. It is a pattern that operates within them, instantiating capabilities that those frameworks describe at higher levels of abstraction. The mapping below is light by design: the goal is to help architects place PAHA inside their existing mental models, not to claim that PAHA reframes the underlying frameworks.

Framework	PAHA Element	Mapping
TOGAF 10th Edition	Governance plane; capability registry	Governance plane corresponds to TOGAF Architecture Governance and reference architectures. Capability registry corresponds to TOGAF capability and architecture catalogs. Three-plane decomposition aligns with Business, Information Systems, and Technology Architecture domains, with the harness as a cross-cutting concern.
ISO/IEC/IEEE 42010:2022	Architectural primitives; governance contracts	Governance contracts are architecture descriptions in 42010 terms. Primitives (contracts, boundaries, adapters, registry) constitute the architecture viewpoint. Three planes correspond to three concerns addressed by distinct viewpoints.
UAF Version 1.2	Meta-harness; cross-cutting	Meta-harness corresponds to UAF cross-cutting

Framework	PAHA Element	Mapping
	services	services and infrastructure viewpoints. Capability registry maps to UAF capability viewpoint. Execution boundary engine corresponds to UAF security and resource viewpoints. The defense IT context makes UAF the most directly applicable framework.
SAFe Enterprise Architect Guidance	Architectural runway; intentional architecture	Harness corresponds to SAFe architectural runway: persistent investment that enables future capability delivery without per-deployment rework. Substrate adapters correspond to SAFe enabler features. Capability-centric framing aligns with SAFe portfolio-level intentional architecture.
Doerr OKR Methodology	Governance contracts; trust escalation model	Trust escalation transitions correspond to OKR-style commitment vs. aspirational target distinctions. Governance contracts specify outcome obligations rather than activity prescriptions, consistent with Doerr's emphasis on outcomes over outputs.

Table 1. Mapping PAHA elements to established enterprise architecture frameworks. The mappings are illustrative rather than canonical; PAHA inhabits these frameworks rather than displacing them.

The most important mapping is the alignment with UAF. Defense IT enterprises that already produce UAF artifacts can integrate PAHA without inventing new viewpoints; the meta-harness is a cross-cutting service expressible in UAF terms, and the capability registry is a UAF capability viewpoint instance. Organizations operating under TOGAF should recognize the harness as a reference architecture in the TOGAF sense and govern it through Architecture Governance processes that already exist. Neither mapping is forced. PAHA was developed within the same intellectual tradition as these frameworks and inherits their conceptual vocabulary by design.

16. Phased Adoption for Organizations with Vendor-Native Deployments

Most organizations that would benefit from PAHA are not greenfield. They already have vendor copilots in production, or have committed to vendor-specific orchestration runtimes through contracts or pilots. A realistic adoption path must accommodate that starting point. The four-phase approach below is calibrated to organizations that have multiple vendor AI deployments in flight and that recognize the resulting fragmentation as a strategic concern.

Phase 0: Baseline the Governance Plane

Before any harness construction, the organization establishes the governance plane as a coherent enterprise function. This is the work that vendor-specific deployment plans label as foundation work: CMDB remediation, governance policy drafting, trust escalation model definition, data sovereignty decisions, sensitivity label deployment, DLP configuration, and audit aggregation. Reframing this work as governance plane construction rather than vendor preparation produces a more durable framing. The governance plane's artifacts will be inherited by every subsequent harness instance and every console built above it. This phase typically requires six to twelve months in defense IT environments and produces no AI capability in itself; it produces the conditions under which AI capability becomes governable.

Phase 1: Introduce Shared Capability Registry and Audit Format

With the governance plane in place, the organization introduces two services that begin to consolidate vendor sprawl: a capability registry that catalogs every AI capability across vendor stacks, and a shared audit format that all vendor stacks emit into. Neither service replaces the vendor stacks at this phase. They are wrappers that produce uniform metadata. The registry tells the organization what AI capabilities exist, who owns them, what governance contracts they operate under, and what substrate they bind to. The audit format ensures that activity from Cisco AI Canvas, AWS Bedrock AgentCore, and Microsoft Copilot Studio is queryable through a single interface. This phase typically requires three to six months and is largely an integration exercise. It does not require vendors to change their behavior; it requires the enterprise to consume vendor outputs uniformly.

Phase 2: Lift a Subset of Agents into a Vendor-Neutral Harness

With the registry and audit format in place, the organization builds or acquires the remaining harness services (gateway, adapter layer, boundary engine, memory governance, console façade) and migrates a small set of agents from vendor-native runtimes into the new harness. The migration target should be agents that benefit most from substrate flexibility (cross-domain agents, agents that need air-gapped variants, agents that should be portable across enclaves). The vendor copilots remain in production for the agents that have not been migrated. This phase establishes the harness as a real artifact serving real workloads, and it generates the operational evidence required to justify further migration. This phase typically requires nine to eighteen months.

Phase 3: New Consoles Are Built on the Harness by Default

Once the harness has demonstrated operational value with migrated agents, the organization changes its default architectural posture: new consoles and new agents are built on the harness rather than on vendor-native runtimes. Vendor copilots may continue to operate for legacy use cases or where their specific capabilities are not yet replicable in the harness. Over time, as the harness's capability surface grows and as agents are progressively migrated, the vendor copilots become a contracting footprint rather than an expanding one. This is the steady state. The organization is not vendor-free; it is vendor-portable.

Substrate decisions become procurement decisions made under enterprise control rather than architectural decisions inherited from vendor selection.

Two warnings about this path. First, organizations that attempt to skip Phase 0 will fail. The governance plane cannot be retrofitted around an in-flight harness; it must precede the harness. Second, organizations that attempt to skip Phase 2 (migrating real agents) and proceed directly from registry/audit consolidation to greenfield harness deployments will produce a paper architecture with no operational evidence. The lifted agents in Phase 2 are what generate the trust required for Phase 3.

17. Conclusion

The future enterprise AI environment will not consist of isolated monolithic assistants for organizations operating under sovereignty constraints, multi-vendor obligations, or cross-enclave security boundaries. For those organizations, the durable architecture will consist of governed, portable, fit-for-purpose agent ecosystems operating through shared meta-harness patterns instantiated per enclave with shared governance artifacts.

In such systems, models become interchangeable through substrate adapters, agents become composable through capability registries, governance becomes centralized through the governance plane, execution becomes bounded through enforcement at the operational plane, and operational consoles become mission-specialized surfaces over a common control substrate. The harness pattern establishes the structural separation necessary to achieve that outcome.

The framework's claims must be qualified in three respects. The harness is portable as a pattern but instantiated per enclave; "one harness" is shorthand for "harness-as-pattern with shared governance artifacts." The framework's value depends on practitioner adoption, which requires that the harness absorb integration burden rather than expose it; governance value is invisible when it works, and friction is the default failure mode. The orchestration supremacy thesis applies most strongly to sovereignty-bounded organizations and should not be over-generalized to commercial contexts where vendor-integrated orchestration may be the more rational choice.

These qualifications strengthen rather than weaken the framework. They specify the conditions under which the architectural pattern delivers value, which is what enterprise architecture frameworks are required to do. The PAHA pattern is not a universal solution. It is a defensible architectural response to a specific class of constraints, and within that class, it is the more durable choice than the assistant-centric alternative.

This paper specifies the architectural pattern. An implementation specification subordinate to a parent enterprise systems engineering management plan is in development and will cite this paper as its architectural framework. That subordinate specification develops the artifact-level realization of the seven minimum viable harness services, the federation mechanics across the enclave constraint, and the engineering processes that produce and maintain harness-conformant AI architectures within an enterprise's existing systems engineering practice. The relationship between the architectural pattern and

the implementation specification is implementation-companion: this paper establishes what to build; the subordinate specification establishes how the enterprise's engineering practice produces it. Future revisions of this paper will cite the implementation specification once it is published.

References

ISO/IEC/IEEE. (2022). *ISO/IEC/IEEE 42010:2022, Systems and software engineering — Architecture description*. International Organization for Standardization.

The Open Group. (2022). *The TOGAF Standard, 10th Edition*. The Open Group.

Object Management Group. (2023). *Unified Architecture Framework (UAF), Version 1.2*. OMG.

Scaled Agile, Inc. (2024). *Enterprise Architect — Extended Guidance*. Scaled Agile Framework.

Doerr, J. (2018). *Measure What Matters: How Google, Bono, and the Gates Foundation Rock the World with OKRs*. Portfolio Penguin.

Appendix A: Deployment View Reference Architecture

This appendix presents a deployment view of the Portable Agent Harness Architecture as a reference for organizations operating across multiple security enclaves. The diagram makes concrete two claims developed in the body of the paper: the seven minimum viable harness services specified in Section 4, and the enclave constraint specified in Section 9. Each enclave instantiates the same harness pattern with the same seven services, but each instance is independently authorized, separately operated, and bounded by the controlling regime of its enclave. Federation occurs at the schema and audit-aggregation layer through a shared governance vocabulary; no operational data crosses enclave boundaries.

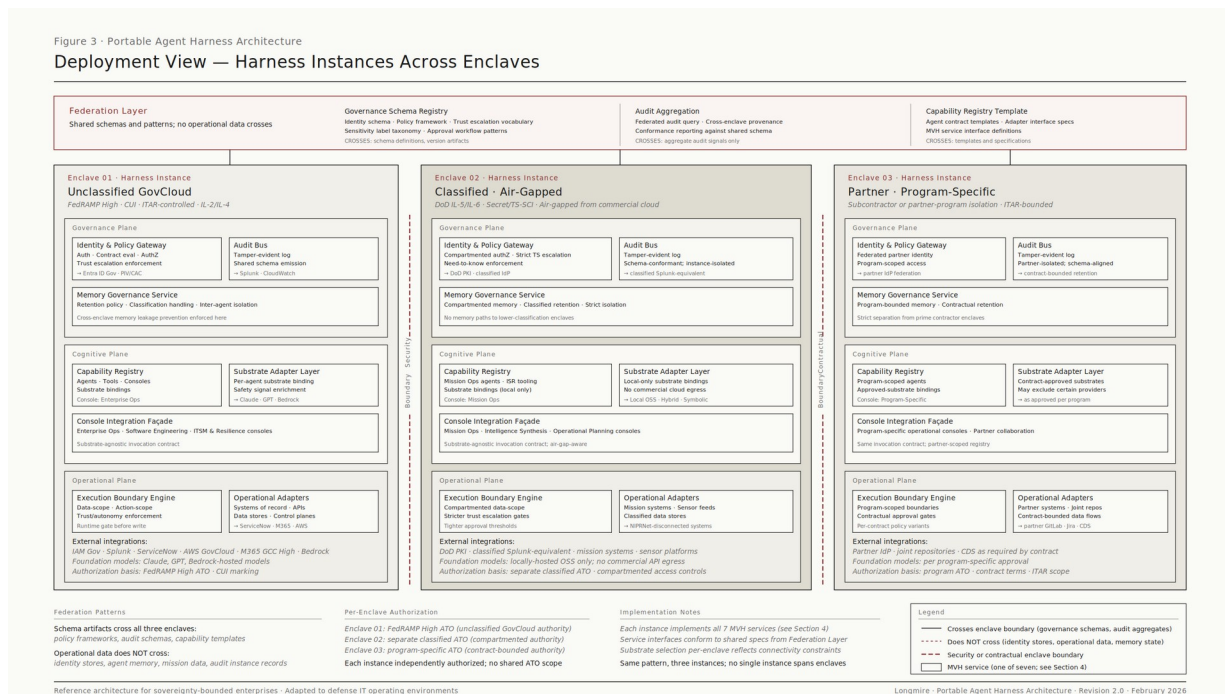


Figure 3. Deployment view of harness instances across three representative enclaves: unclassified GovCloud, classified air-gapped, and partner program-specific. Each enclave contains a complete harness instance with all seven minimum viable harness services (governance plane, cognitive plane, operational plane). The federation layer at the top spans all three enclaves and shares only governance schemas, audit aggregates, and capability registry templates. Boundary lines between enclaves indicate security or contractual separation. Authorization basis, foundation model availability, and external integrations vary per enclave; service interfaces and governance contracts conform to shared specifications.

The reference architecture is illustrative rather than canonical. Specific authorization tags (FedRAMP High, DoD IL-5/IL-6, program-specific ATO), substrate selections (Claude, GPT, Bedrock for unclassified; locally-hosted OSS for classified; per-program approval for partner), and external integration profiles will vary by organization and program. The structural relationships, however, are invariant under PAHA: every enclave instantiates the same seven services, every service interface conforms to a federation-layer specification, and federation transmits patterns rather than data. The federation layer itself is an engineered artifact

rather than an emergent property; the governance schema registry, the audit aggregation capability, and the capability registry template service must be built, owned, governed, and maintained as deliberate enterprise commitments. Treating the federation layer as something that arises naturally from shared inheritance is one of the failure modes that distinguishes paper architectures from operational ones.

Three observations are worth emphasizing for architects evaluating this reference. First, the diagram demonstrates that harness instantiation is not optional. An organization that builds only one harness instance and attempts to extend it across security boundaries has not implemented PAHA; it has implemented a single-enclave assistant pattern with cross-boundary aspirations. Second, the federation layer is a real artifact requiring governance investment of its own; without a governance schema registry, audit aggregation capability, and capability registry template service, the per-enclave instances drift apart and the pattern collapses into independent silos. Third, the per-enclave authorization basis must remain separate. Each instance carries its own ATO, its own compliance posture, and its own operational boundary. Federation exchanges schemas, not authority.

This paper is unaffiliated research. Views expressed are the author's and do not represent the position of any employer or sponsoring organization.